Computer Organization and Assembly Language Lab 01 (clo1)

Demonstrator: Saba Latif
Google class id: gb2cyldx
Session: BSCS 4B

Assembly Language

Advantages of Assembly Language

1. Shows how program interfaces with the processor, operating
system, and BIOS.
2. Shows how data is represented and stored in memory and on external devices.
3. Clarifies how processor accesses and executes instructions and how instructions access and process data.
4. Clarifies how a program accesses external devices.

Assembly Language

Reasons for using Assembly Language

1. A program written in Assembly Language requires considerably less memory and execution time than one written in a high –level language.

2. Assembly Language gives a programmer the ability to perform highly technical tasks that would be difficult, if not impossible in a high-level language.

Assembly Language

Software tools are needed for editing, assembling, linking, and debugging assembly language programs:

Editor:

Allows you to create assembly language source files.

Assembler:

A program that converts source-code programs written in assembly language into object files in machine language.

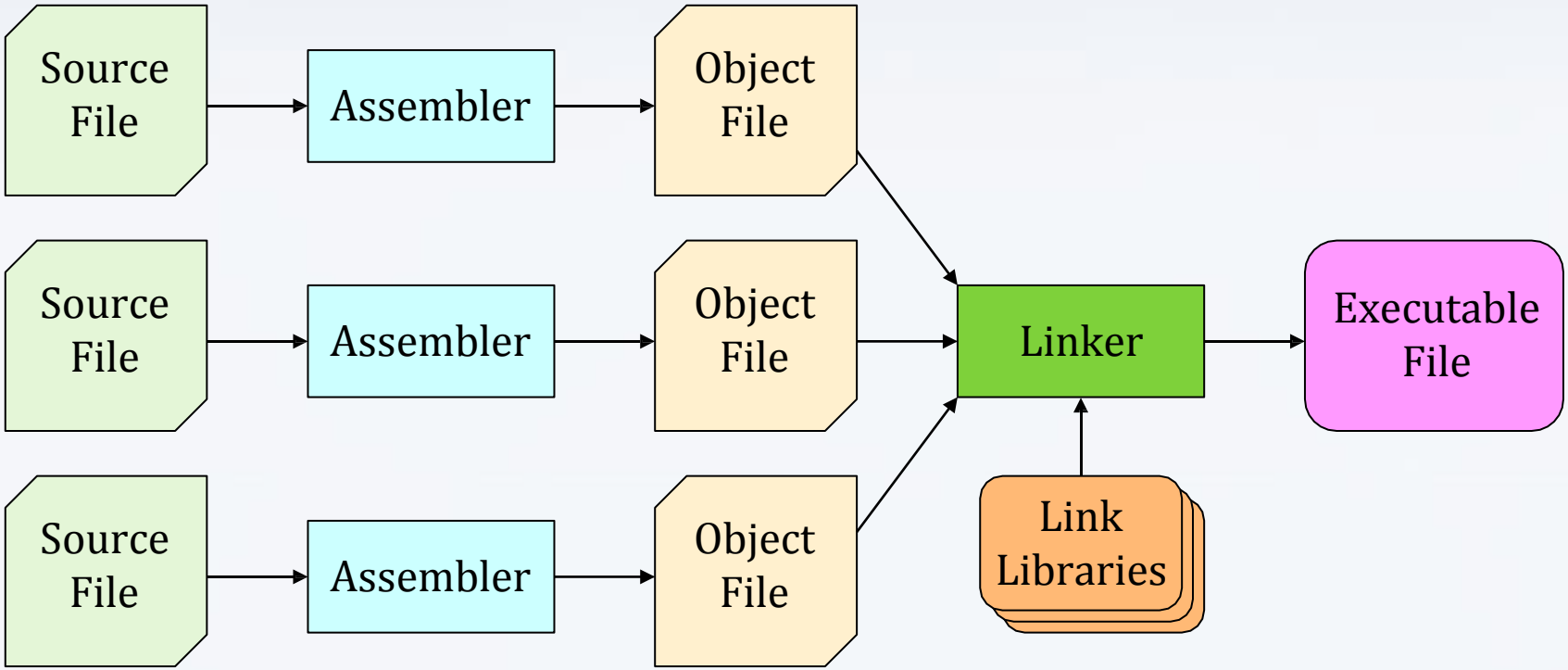
Assembly Language

Software tools are needed for editing, assembling, linking, and debugging assembly language programs:

Linker:

It combines your program's object file created by the assembler with other object files and link libraries, and produces a single executable program.

Assembly Language



- A project may consist of multiple source files.
- Assembler translates each source file separately into an object file.
- Linker links all object files together with link libraries.

Sim#

- **QtSpim**Sim# is a desktop application running in a Windows environment.
- It allows users to simulate the execution of **QtSpim** assembly language programs on a system based on the MIPS processor.
- **QtSpim**Sim# includes both an assembler and a linker; when a file is loaded, the simulator automatically assembles and links the program.

QtSpim Program Formats

- The QtSpim simulator has a number of features and requirements for writing MIPS assembly language programs. This includes a properly formatted assembly source file. A properly formatted assembly source file consists of two main parts;

the data section (where data is placed) and the text section (where code is placed).

Comments

- The "#" character represents a comment line. Anything typed after the "#" is considered a comment.

Assembler Directives

- Directives are required for data declarations and to define the start and end of procedures. Assembler directives start with a "." For example, ".data" or ".text".
- Additionally, directives are used to declare and defined data. The following sections provide some examples of data declarations using the directives.

Data Declarations

- The data must be declared in the ".data" section.
- All variables and constants are placed in this section.
- Variable names must start with a letter followed by letters or numbers
- (including some special characters such as the "_"), and terminated with a ":" (colon).
- Variable definitions must include the name, the data type, and the initial value for the variable. In the definition, the variable name must be terminated with a ":".
- The data type must be preceded with a "." (period). The general format is: **<variableName>: .<dataType> <initialValue>**

Refer to the following sections for a series of examples using various data types.

The supported data types are as follows:

These are the primary assembler directives for data declaration

Declaration	
`.byte`	8-bit variable(s)
`.half`	16-bit variable(s)
`.word`	32-bit variable(s)
`.ascii`	ASCII string
`.asciiz`	NULL terminated ASCII string
`.float`	32 bit IEEE floating point number
`.double`	64 bit IEEE floating point number
`.space <n>`	<n> bytes of uninitialized memory

Integer Data Declarations

- Integer values are defined with the *.word, .half, or .byte directives.*
- *Two's compliment is* used for the representation of negative values

The following declarations are used to define the integer variables "wVar1" and "wVar2" as 32-bit word values and initialize them to 500,000 and -100,000.

```
wVar1:      .word       500000
wVar2:      .word       -100000
```

The following declarations are used to define the integer variables "hVar1" and "hVar2" as 16-bit word values and initialize them to 5,000 and -3,000.

```
hVar1:      .half       5000
hVar2:      .half       -3000
```

The following declarations are used to define the integer variables "bVar1" and "bVar2" as 8-bit word values and initialize them to 5 and -3.

```
bVar1:      .byte       5
bVar2:      .byte       -3
```

If an variable is initialized to a value that can not be stored in the allocated space, an assembler error will be generated. For example, attempting to set a byte variable to 500 would be illegal and generate an error.

String Data Declarations

- Strings are defined with *.ascii or .asciiz directives. Characters are represented using* standard ASCII characters.

The following declarations are used to define a string "message" and initialize it to "Hello World".

```
message:    .asciiz     "Hello World\n"
```

In this example, the string is defined as NULL terminated (i.e., after the new line). The NULL is a non-printable ASCII character and is used to mark the end of the string. The NULL termination is standard and is required by the print string system service (to work correctly).

To define a string with multiple lines, the NULL termination would only be required on the final or last line. For example,

```
message:    .ascii      "Line 1: Goodbye World\n"
            .ascii      "Line 2: So, long and thanks "
            .ascii      "for all the fish.\n"
            .asciiz     "Line 3: Game Over.\n"
```

Floating-Point Data

Floating-point values are defined with the *.float* (32-bit) or *.double* (64-bit) directives. The IEEE floating-point format is used for the internal representation of floating-point values.

The following declarations are used to define the floating-point variables "pi" to a 32-bit floating-point value initialized to 3.14159 and "tao" to a 64-bit floating-point values initialized them to 6.28318.

```
pi:         .float      3.14159
tao:        .double     6.28318
```

Constants

Constant names must start with a letter followed by letters or numbers including some special characters such as the "_" (underscore). Constant definitions are created with an "=" sign.

For example, to create some constants named TRUE and FALSE and set them to 1 and 0 respectively:

```
TRUE = 1
FALSE = 0
```

programmers more easily distinguish between variables (which can change values) constants (which can not change values). Additionally, in assembly language cons are not typed (i.e., not predefined to be a specific size such as 8-bits, 16-bits, 32-bi 64-bits).

Constants are also defined in the data section. The use of all capitals for a constant is a convention and not required by the QtSpim program. The convention helps

Program Code

- The code must be preceded by the ".text" directive.
- The ".globl name" and ".ent name" directives are required to define the
  name of the initial or main procedure.
- The main procedure (as all procedures) should be terminated with the ".end <name>" directive.

Labels

Labels are code locations, typically used as function/procedure name or as the target of a jump.
The first use of a label is the main program starting location, which must be
named '**main' which is a specific requirement for the QtSpim simulator.**

Label

The rules for a label are as follows:

- Must start with a letter
- May be followed by letters, numbers, or an "_" (underscore).
- Must be terminated with a ":" (colon).
- May only be define once.

Some examples of a label include:

```
main:
exitProgram:
```

Characters in a label are case-sensitive. As such, **`Loop:`** and **`loop:`** are different labels. This can be very confusing initially, so caution is advised.

Program Template

- The following is a very basic template for QtSpim MIPS programs. This general template will be used for all programs.

```
#_____________________________
# Data declarations go in this section.
    .data
# program specific data
declarations
#_____________________________
#Program code goes in this section.
      .text
      .globl main
      .ent  main
      main:
# your program code goes here.
#_____________________________
#Done, terminate
       program. li $v0, 10
       syscall
      .end main
```

Downloading and Installing QtSpim

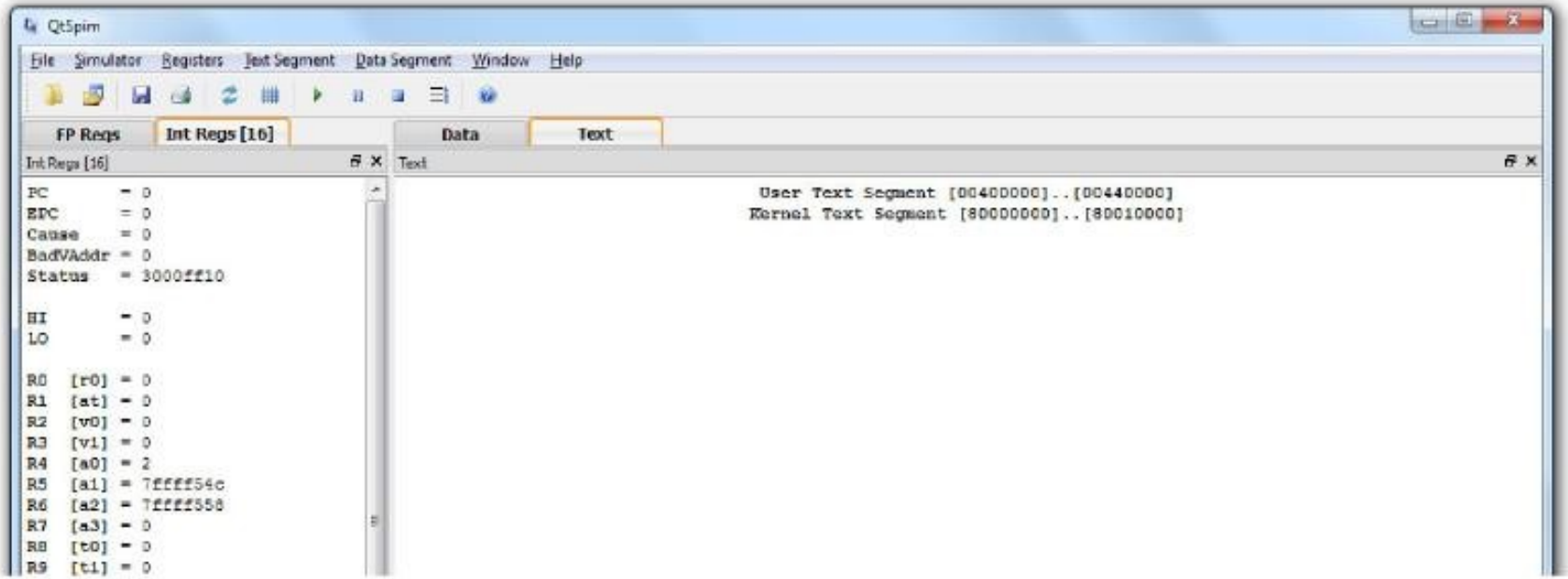
- The following are the current URLs for QtSpim.
- The QtSpim home page is located at: http://spimsimulator.sourceforge.net/
- The specific download site is located at: http://sourceforge.net/projects/spimsimulator/files/
- At the download site there are multiple versions for different target machines.

  These include Windows (all versions), Linux (32-bit), Linux (64-bit), and Mac OS (all versions). Download the latest version for your machine.

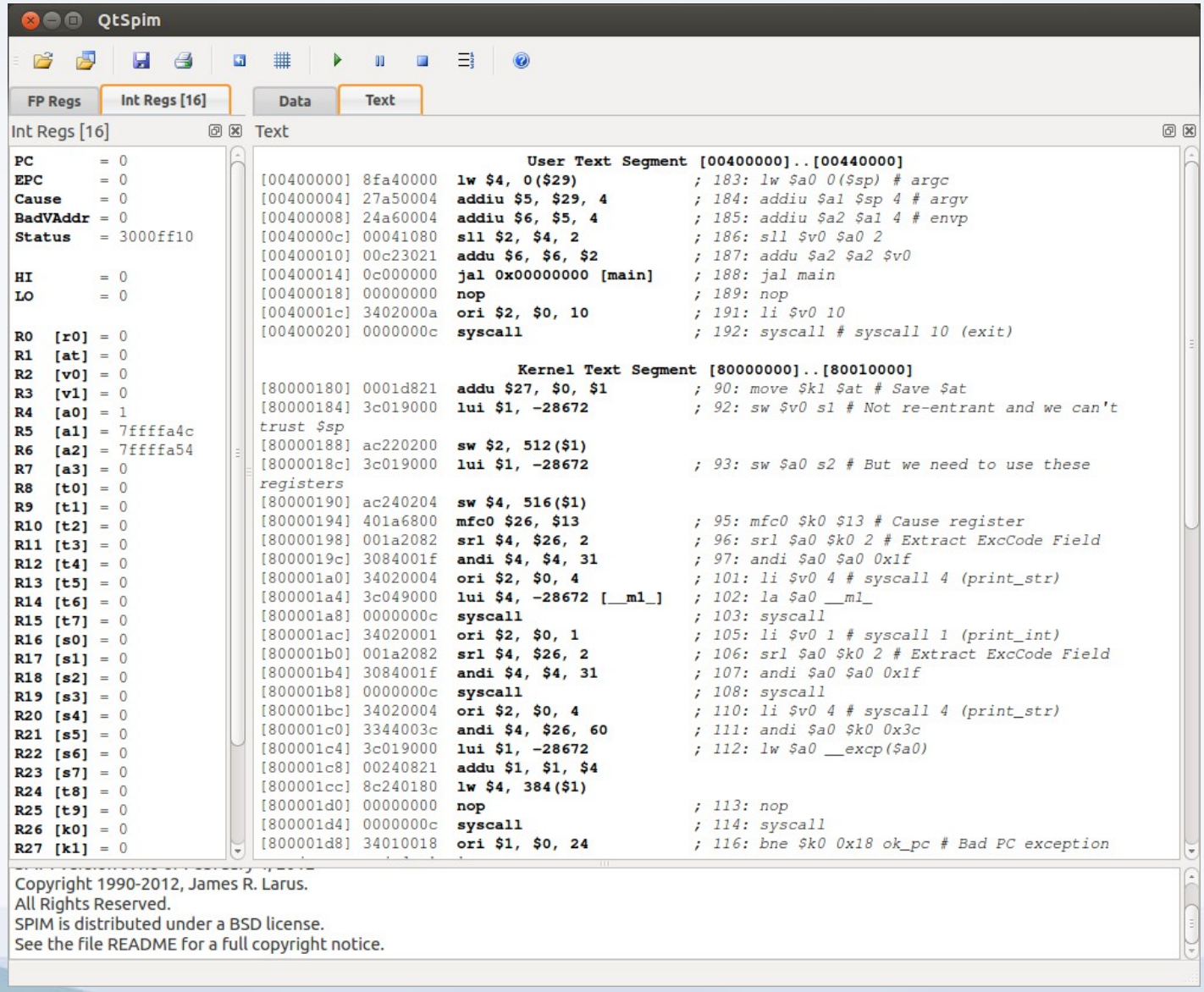
Starting QtSpim

- For Windows, this is typically, performed with the standard

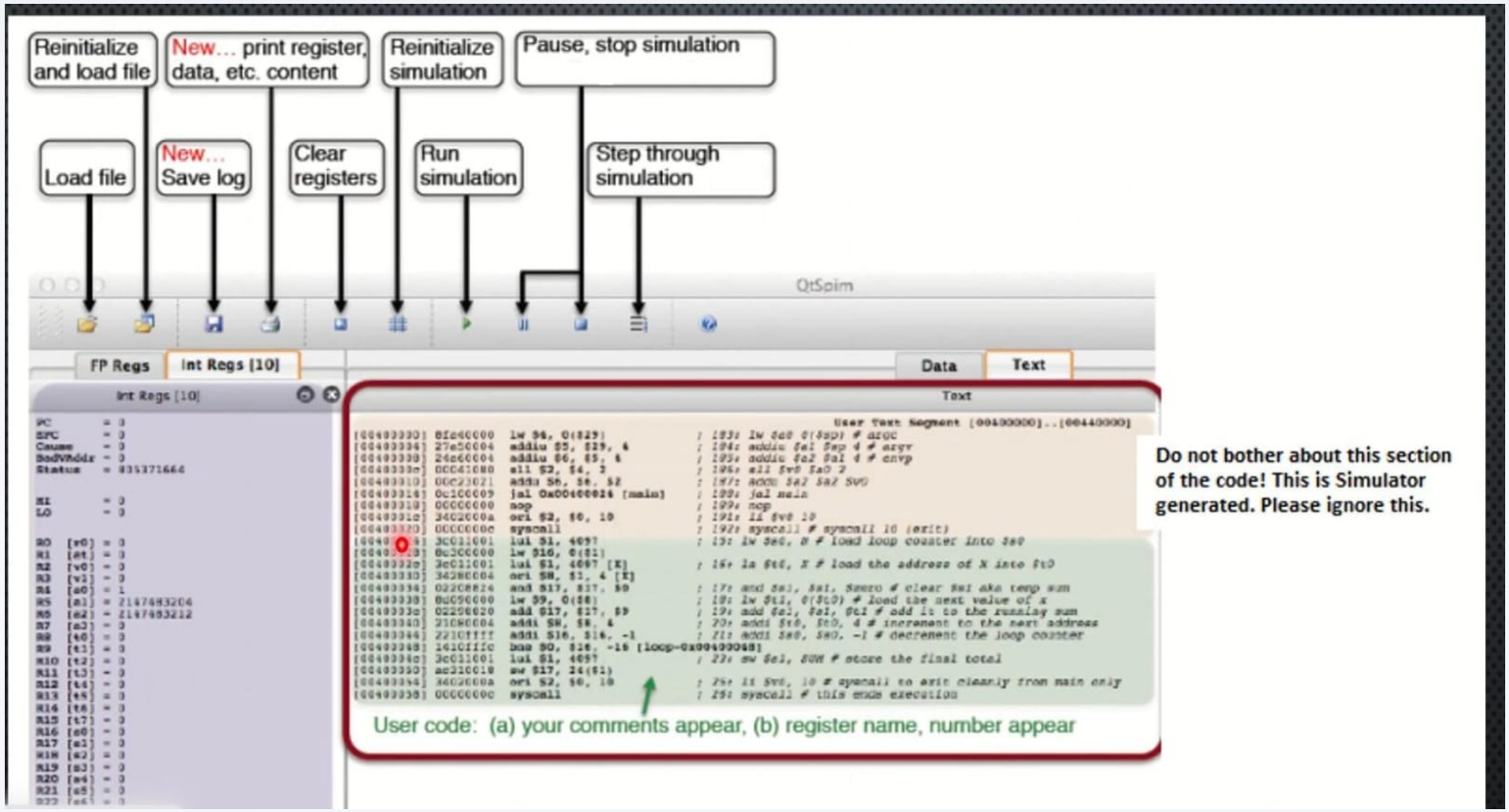
"Start Menu -> Programs -> QtSpim "



Main


QtSpim
FP Regs
Int Regs [16]
Data
Text
Int Regs [16]
PC = 0
EPC = 0
Cause = 0
BadVAddr = 0
Status = 3000ff10
HI = 0
LO = 0
R0 [r0] = 0
R1 [at] = 0
R2 [v0] = 0
R3 [v1] = 0
R4 [a0] = 1
R5 [a1] = 7ffffa4c
R6 [a2] = 7ffffa54
R7 [a3] = 0
R8 [t0] = 0
R9 [t1] = 0
R10 [t2] = 0
R11 [t3] = 0
R12 [t4] = 0
R13 [t5] = 0
R14 [t6] = 0
R15 [t7] = 0
R16 [s0] = 0
R17 [s1] = 0
R18 [s2] = 0
R19 [s3] = 0
R20 [s4] = 0
R21 [s5] = 0
R22 [s6] = 0
R23 [s7] = 0
R24 [t8] = 0
R25 [t9] = 0
R26 [k0] = 0
R27 [k1] = 0
Text
User Text Segment [00400000]..[00440000]
[00400000] 8fa40000 lw $4, 0($29) ; 183: lw $a0 0($sp) # argc
[00400004] 27a50004 addiu $5, $29, 4 ; 184: addiu $a1 $sp 4 # argv
[00400008] 24a60004 addiu $6, $5, 4 ; 185: addiu $a2 $a1 4 # envp
[0040000c] 00041080 sll $2, $4, 2 ; 186: sll $v0 $a0 2
[00400010] 00c23021 addu $6, $6, $2 ; 187: addu $a2 $a2 $v0
[00400014] 0c000000 jal 0x00000000 [main] ; 188: jal main
[00400018] 00000000 nop ; 189: nop
[0040001c] 3402000a ori $2, $0, 10 ; 191: li $v0 10
[00400020] 0000000c syscall ; 192: syscall # syscall 10 (exit)
Kernel Text Segment [80000000]..[80010000]
[80000180] 0001d821 addu $27, $0, $1 ; 90: move $k1 $at # Save $at
[80000184] 3c019000 lui $1, -28672 ; 92: sw $v0 s1 # Not re-entrant and we can't trust $sp
[80000188] ac220200 sw $2, 512($1)
[8000018c] 3c019000 lui $1, -28672 ; 93: sw $a0 s2 # But we need to use these registers
[80000190] ac240204 sw $4, 516($1)
[80000194] 401a6800 mfc0 $26, $13 ; 95: mfc0 $k0 $13 # Cause register
[80000198] 001a2082 srl $4, $26, 2 ; 96: srl $a0 $k0 2 # Extract ExcCode Field
[8000019c] 3084001f andi $4, $4, 31 ; 97: andi $a0 $a0 0x1f
[800001a0] 34020004 ori $2, $0, 4 ; 101: li $v0 4 # syscall 4 (print_str)
[800001a4] 3c049000 lui $4, -28672 [__m1_] ; 102: la $a0 __m1_
[800001a8] 0000000c syscall ; 103: syscall
[800001ac] 34020001 ori $2, $0, 1 ; 105: li $v0 1 # syscall 1 (print_int)
[800001b0] 001a2082 srl $4, $26, 2 ; 106: srl $a0 $k0 2 # Extract ExcCode Field
[800001b4] 3084001f andi $4, $4, 31 ; 107: andi $a0 $a0 0x1f
[800001b8] 0000000c syscall ; 108: syscall
[800001bc] 34020004 ori $2, $0, 4 ; 110: li $v0 4 # syscall 4 (print_str)
[800001c0] 3344003c andi $4, $26, 60 ; 111: andi $a0 $k0 0x3c
[800001c4] 3c019000 lui $1, -28672 ; 112: lw $a0 __excp($a0)
[800001c8] 00240821 addu $1, $1, $4
[800001cc] 8c240180 lw $4, 384($1)
[800001d0] 00000000 nop ; 113: nop
[800001d4] 0000000c syscall ; 114: syscall
[800001d8] 34010018 ori $1, $0, 24 ; 116: bne $k0 0x18 ok_pc # Bad PC exception
Copyright 1990-2012, James R. Larus.
All Rights Reserved.
SPIM is distributed under a BSD license.
See the file README for a full copyright notice.

Structure :

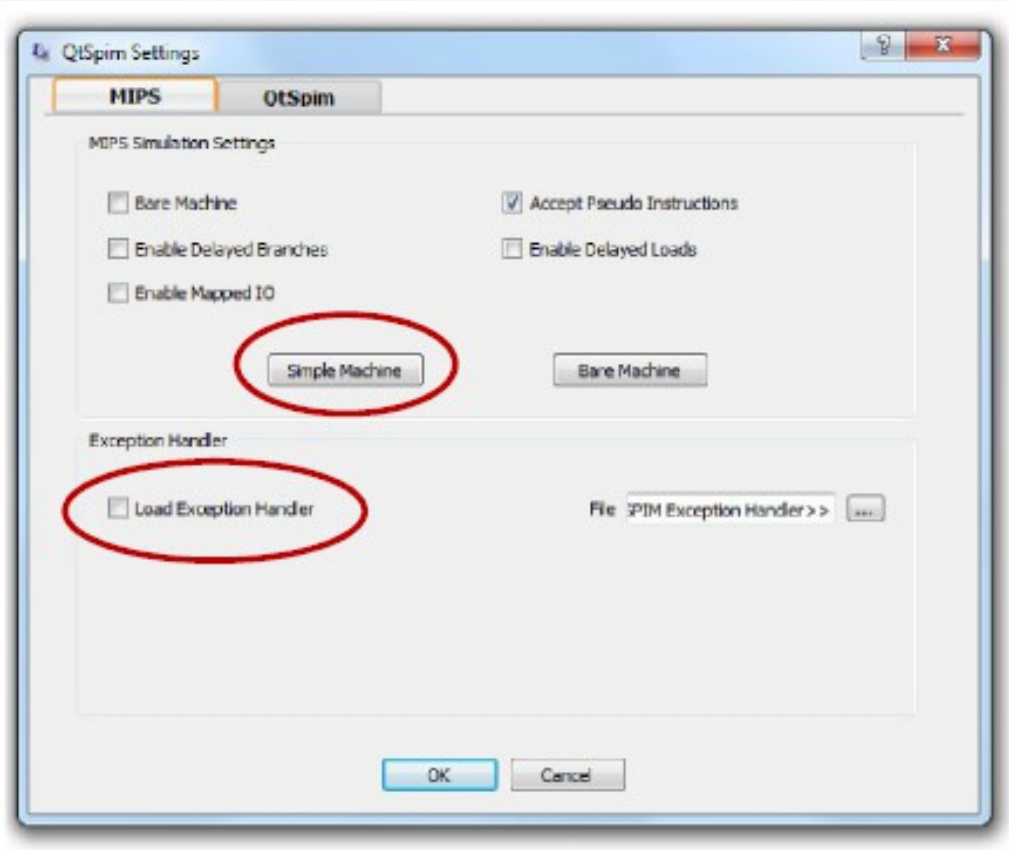

Reinitialize and load file
New... print register, data, etc. content
Reinitialize simulation
Pause, stop simulation
Load file
New... Save log
Clear registers
Run simulation
Step through simulation
QtSpim
FP Regs
Int Regs [10]
Data
Text
Int Regs [10]
Text
User Text Segment [00400000]..[00440000]
Do not bother about this section of the code! This is Simulator generated. Please ignore this.
User code: (a) your comments appear, (b) register name, number appear

check that we have the right simulator configuration for the labs.

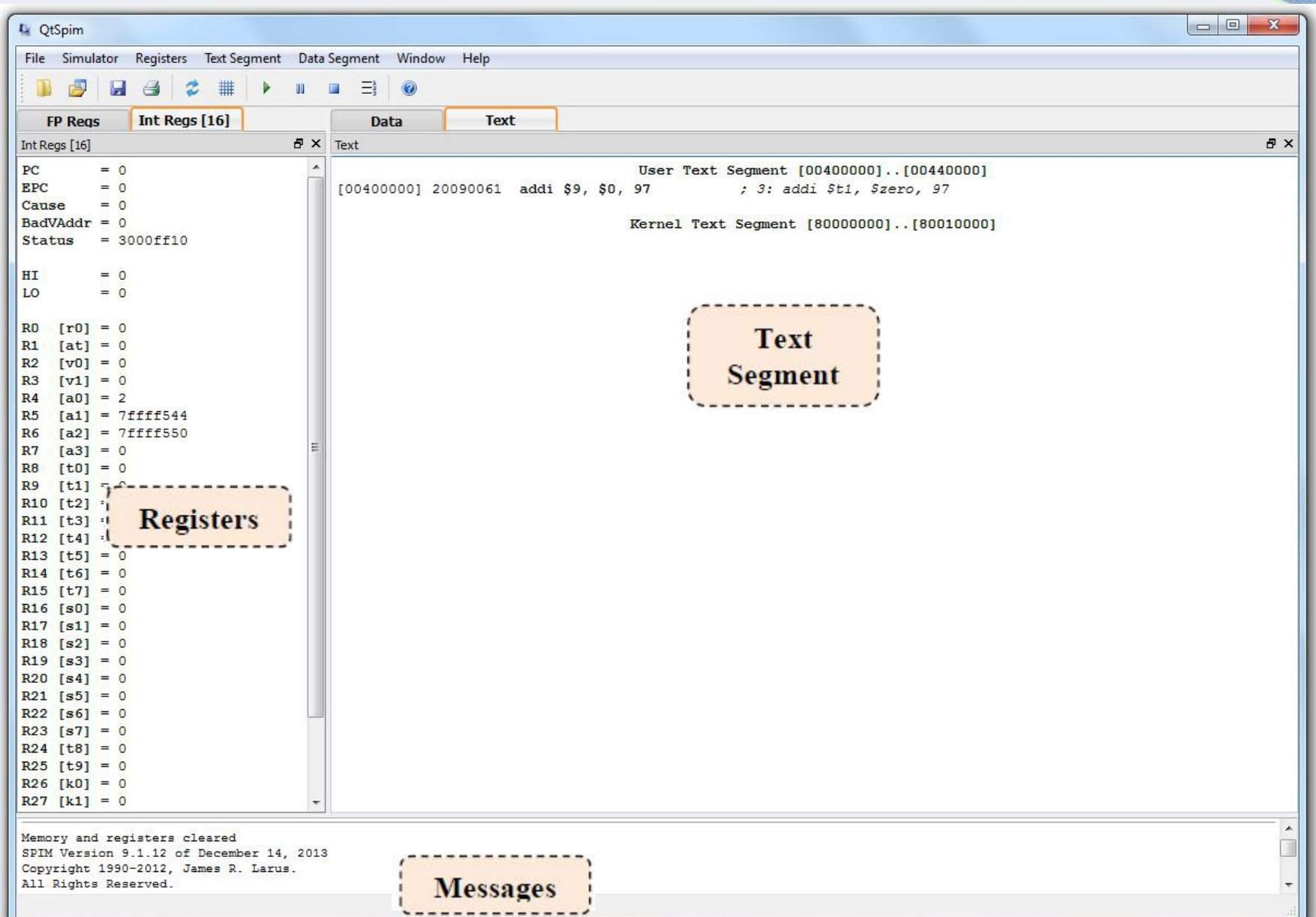
- Click on “Simulator Settings” on the menu bar. Go to the “MIPS” tab then:

1. Click the “Simple Machine” button to use the simplest setting for the processor simulation.
2. Uncheck the “Load Exception Handler”.

Click "OK" to save your setting.



QtSpim window should look exactly the same as follows:



The display is divided into three frames: *Registers*, *Text Segment* and *Messages*.

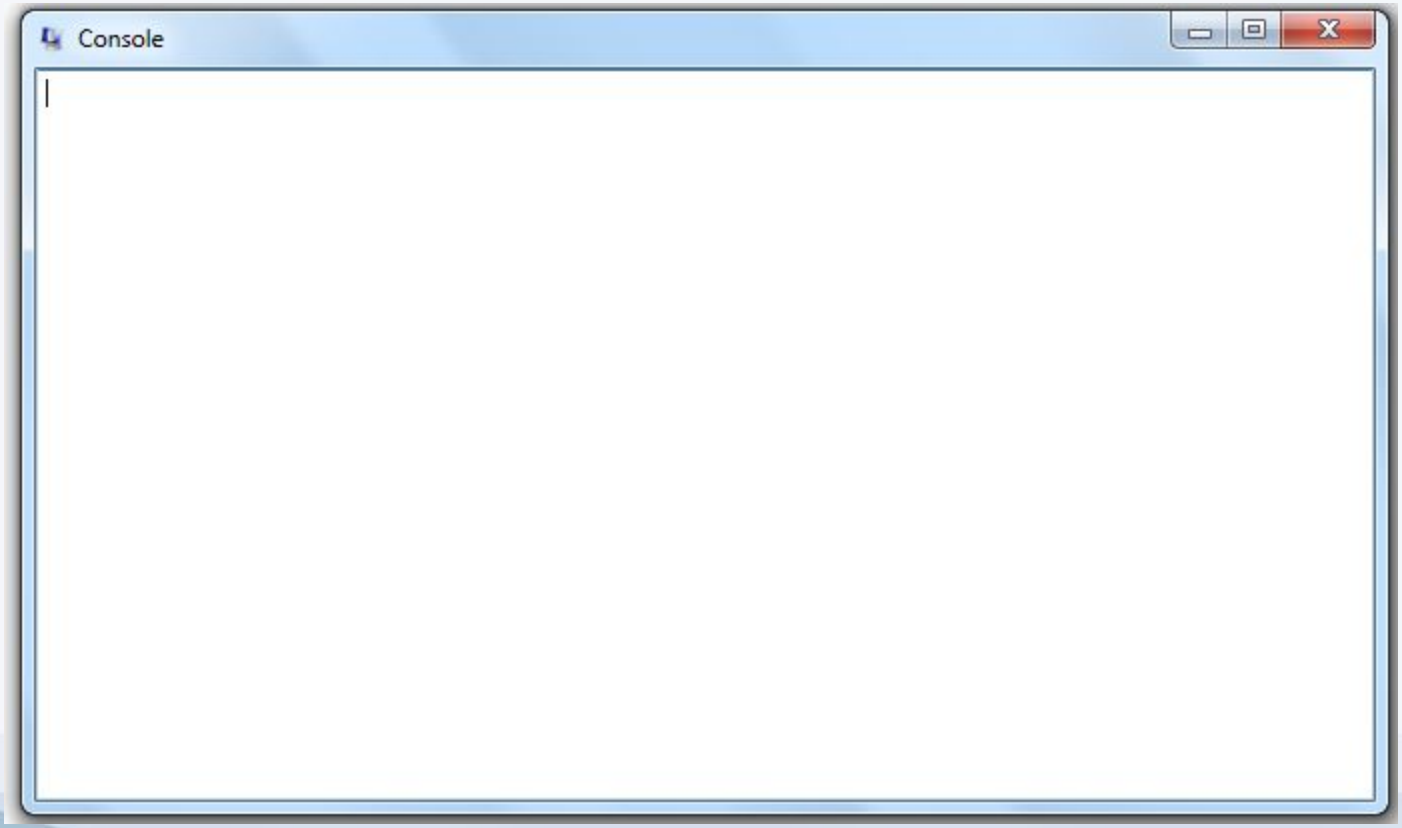
- *Registers frame:* *This window shows the values of special and integer registers in the* MIPS CPU. There is a tab for FPU (floating point unit) registers.
- *Text Segment frame:* *This window displays instructions from your assembly program.*
- *Messages frame:* *This is where QtSpim writes messages and where error messages* appear.

- Note that the *Text Segment Frame can change to display the Data Segment by clicking on the* "Data" tab.

The data segment represents the data your program loaded into the "main memory"

If a program performs input and/or output, it will be directed to the Console window.

- there is also a separate *Console window for input and output. If the* Console window is hidden, you can click "Windows Console" to enable it.

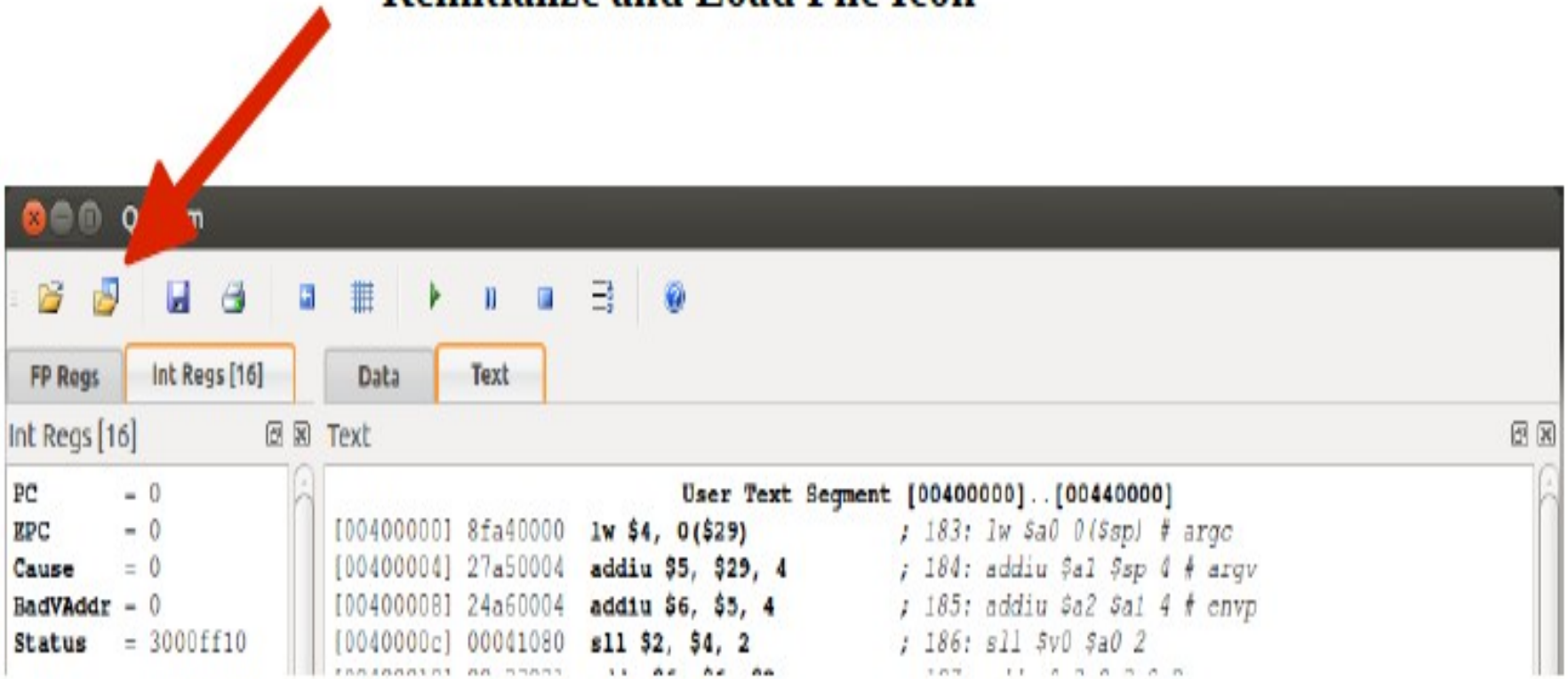


Running the First QtSpim Assembly Language Program

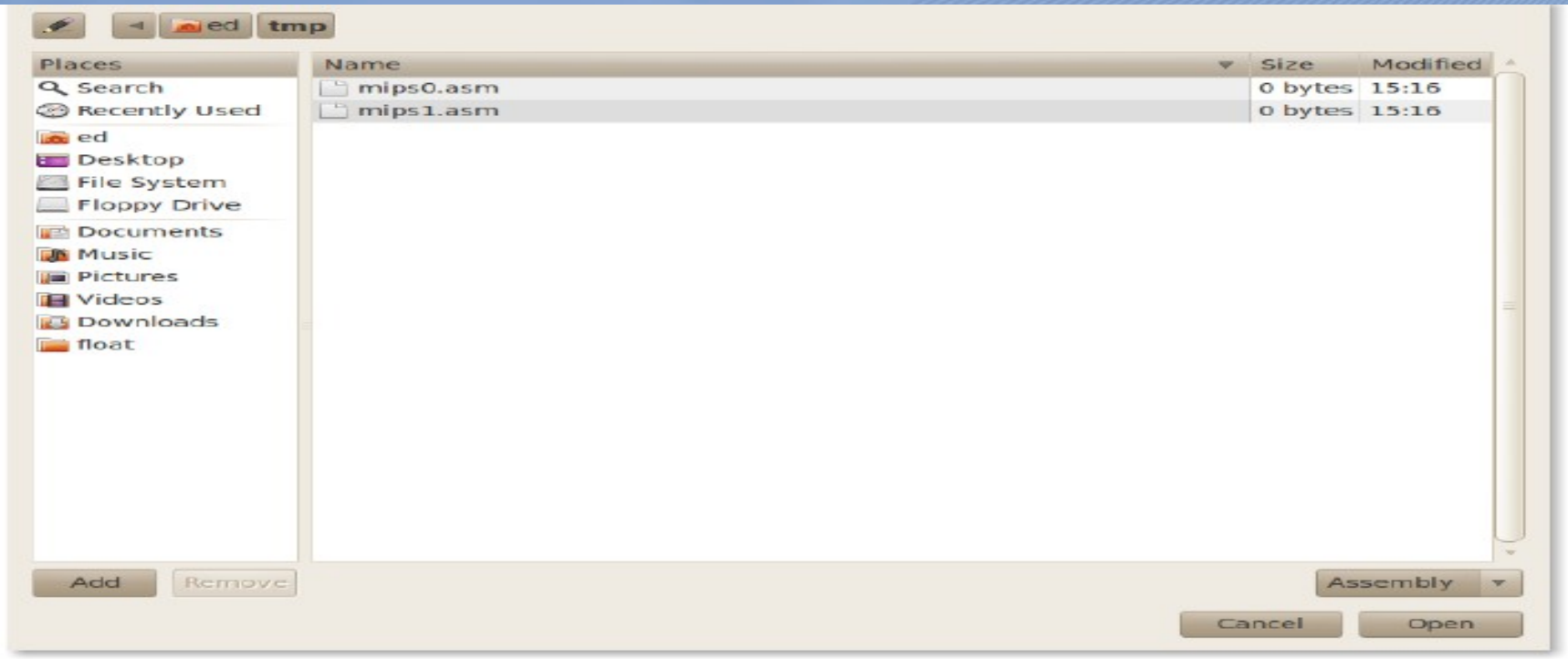
- Follow the stepsbelow to run program in QtSpim Sim#:

1. Write the program in note pad and save as *.s * .o *.asm
2. Open the simulator and select File.
3. Choose Load File option can be used on the initial load, but subsequent file loads will need to use the Reinitialize and Load File to ensure the appropriate reinitialization occurs.
4. You can view the content of the registers that you used in the program in the left side of your working window or register view area.

Reinitialize and Load File Icon



Once selected, a standard open file dialog box will be displayed. Find and select 'asst0.asm' file (or whatever you named it) created in section 3.0.



Navigate as appropriate to find the example file previously created. When found, select the file (it will be highlighted) and click Open button (lower right hand corner).

The assembly process occurs as the file is being loaded. As such, any assembly syntax errors (i.e., misspelled instructions, undefined variables, etc.) are caught at this point. An appropriate error message is provided with a reference to the line number that caused the error. When the file load is completed with no errors, the program is ready to run.

Let us now take a look at the **sample1.asm**:

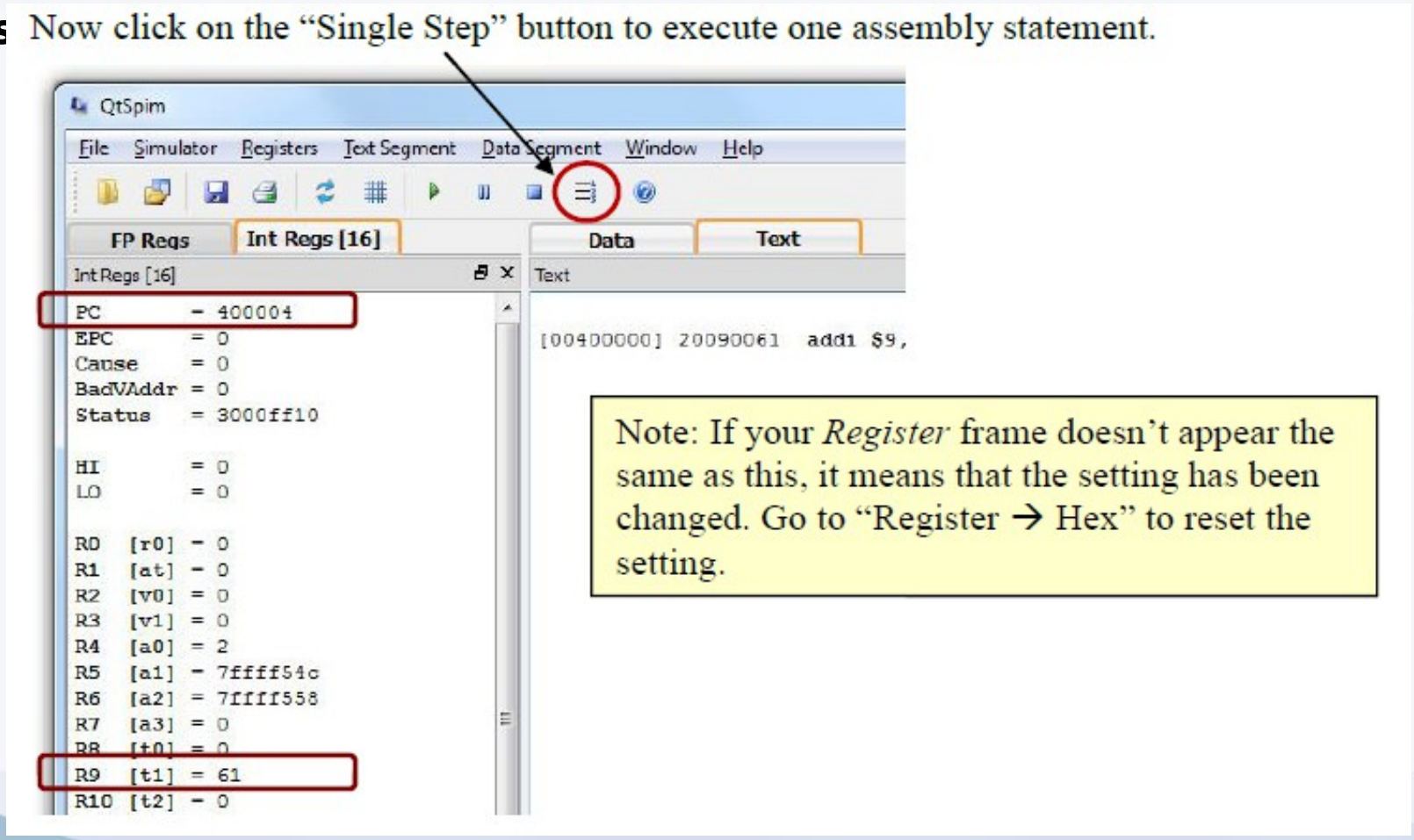
```
# sample1.asm
        .text
main:   addi $t1, $zero, 97
        li $v0, 10
        syscall
```

```
addi $9, $0, 97 ; 3: addi $t1, $zero, 97
ori $2, $0, 10  ; 4: li $v0, 10
syscall         ; 5: syscall
```

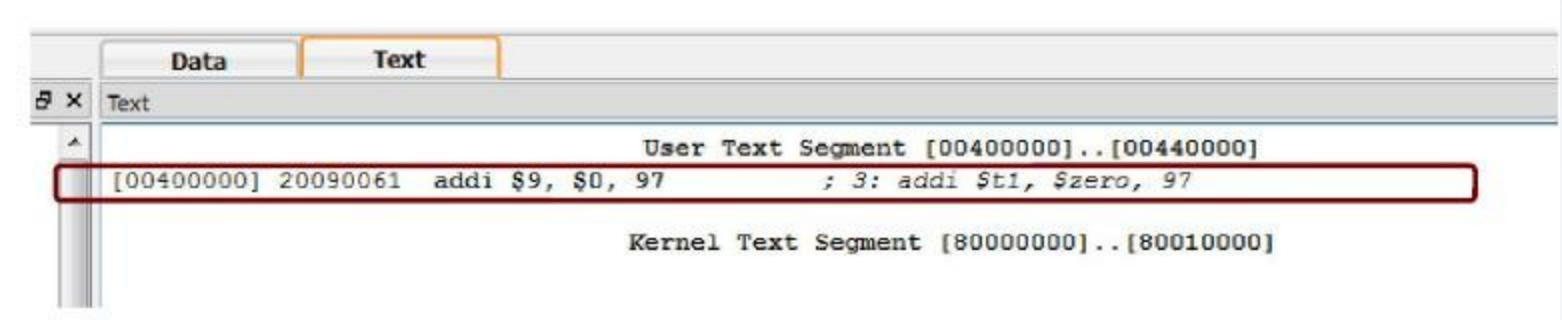
- The first line is a comment line, which begins with "#".
- The second line .text is the *assembler directive that specifies the starting address of the source code.*
- *If no starting* address is specified, the default value 0x00400000 will be assumed.
- The next line contains a MIPS instruction
- the registers $t1 and $zero are translated into their respective numbered registers
  $9 and $0.
- The next two lines li $v0, 10 and syscall constitute a system call to exit.

If you want to make any modification to sample1.asm, you can use text
editor NotePad .

After modification, you need to reload it with "File Load File". Now click on the "Single Step" button to execute one as



Text Segment

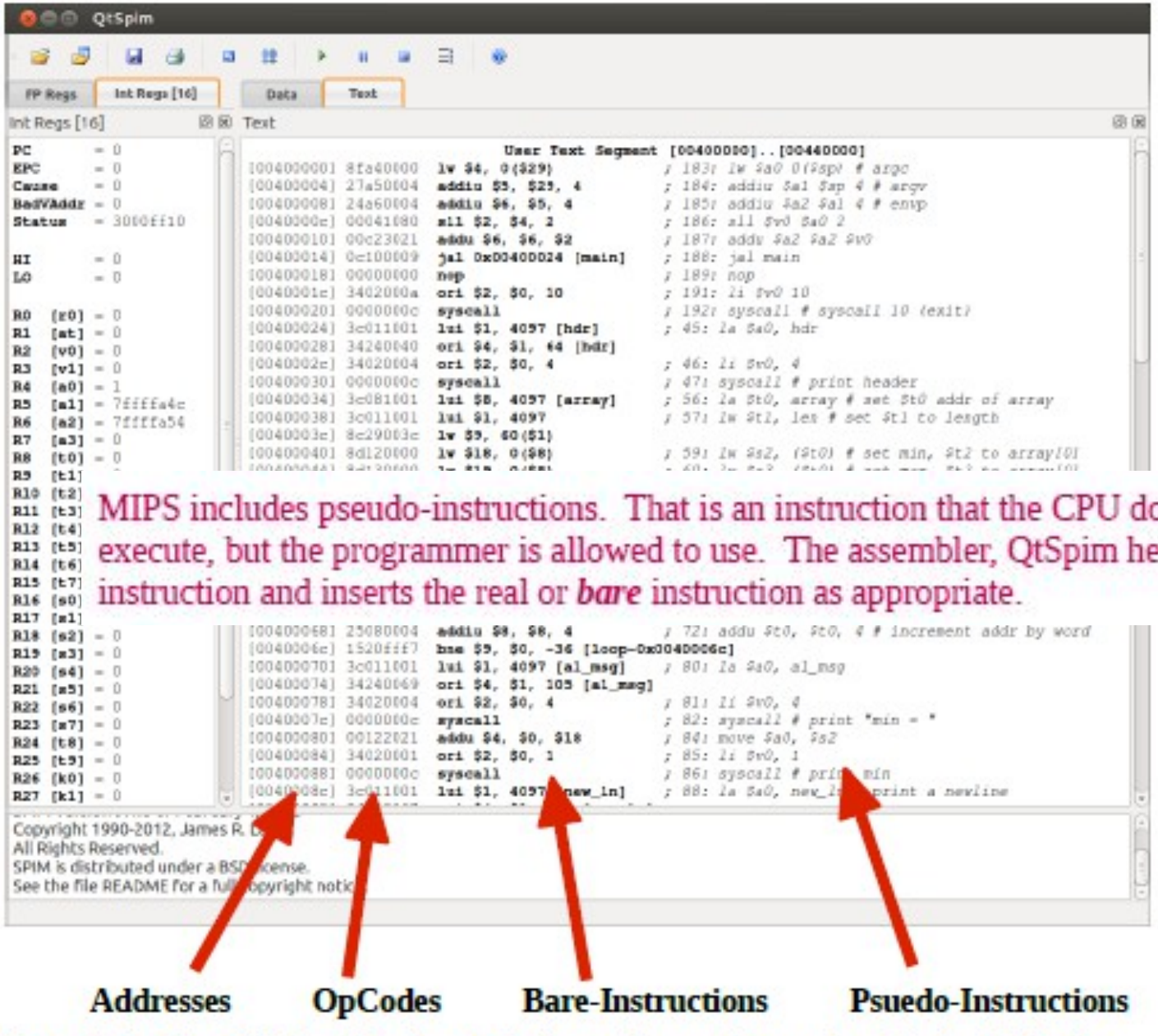


For each line,

1. the first column – [0x00400000] – is the memory location (address) of the instruction;
2. the second column – 0x20090061 – is the encoded instruction in hexadecimal;
3. the third column – addi $9, $0, 97 – is the native code;
4. the fourth column – addi $t1, $zero, 97 – is your source code. Everything following the semicolon is the actual line from your source code.

The number "3:" refers to the line number the corresponding MIPS instruction in the source code.

Note that the first line of your source code begins at address 0x00400000 by default.



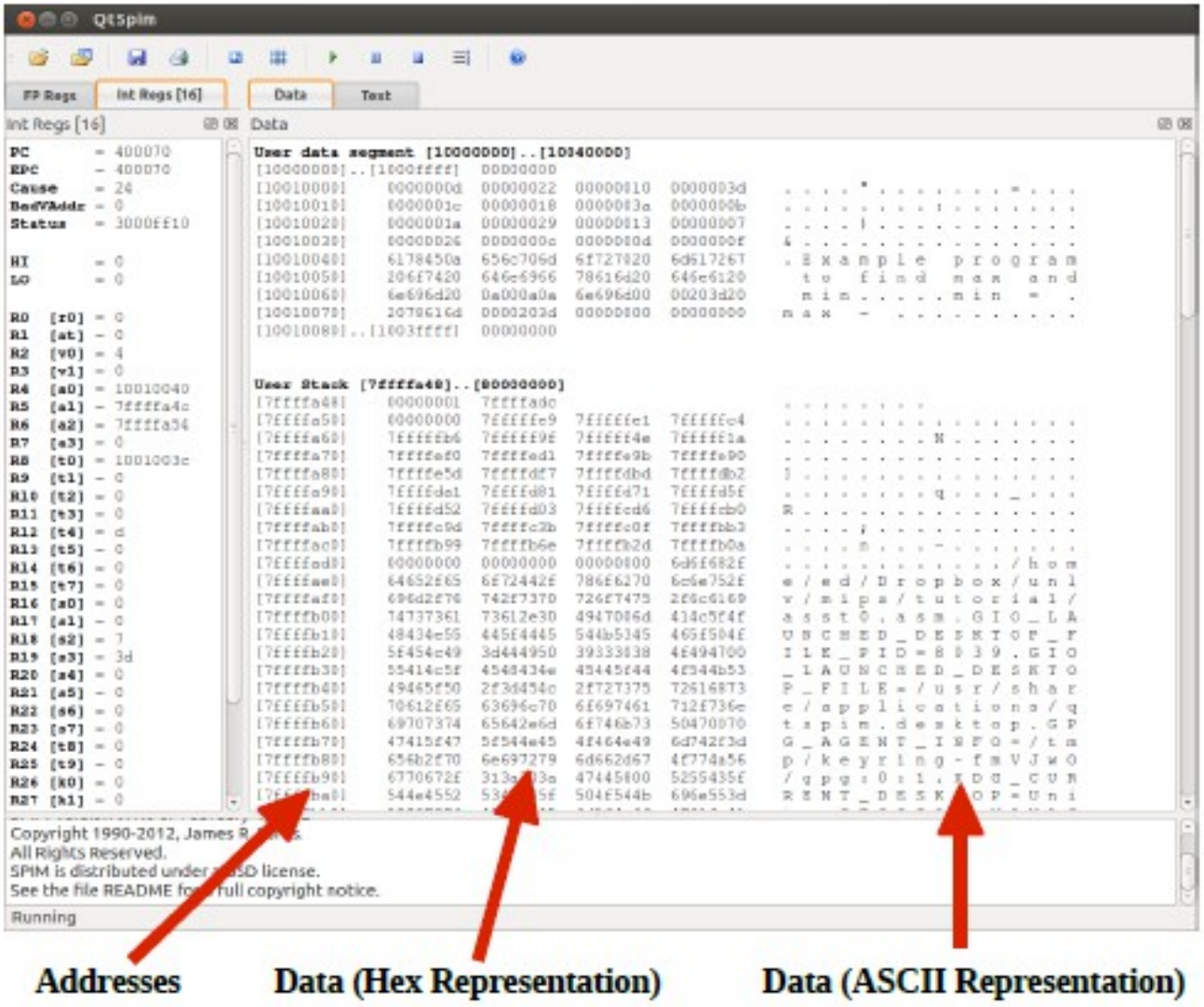
MIPS includes pseudo-instructions. That is an instruction that the CPU does not execute, but the programmer is allowed to use. The assembler, QtSpim here, accepts t instruction and inserts the real or ***bare*** instruction as appropriate.

Addresses **OpCodes** **Bare-Instructions** **Psuedo-Instructions**

The code is placed in Text Window. The first column of hex values (in the []'s) is the address of that line of code. The next hex value is the OpCode or hex value of the 1's and 0's that the CPU understands to be that instruction.

Data

The data segment contains the data declared by your program (if any). To view the data segment, click on the Data Icon. The data window will appear similar to the following:



Running and Stepping through Code

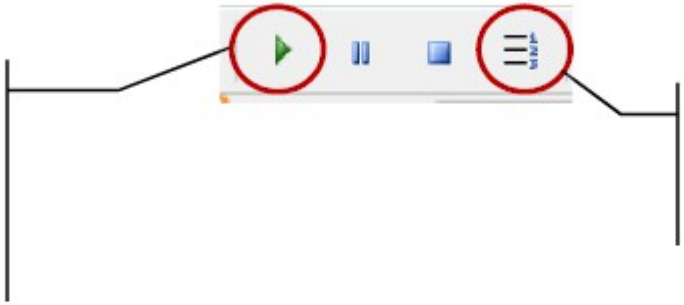
Program Execution

To execute the entire program (uninterrupted), you can select the standard "**Simulator → Run/Continue**" option from the menu bar. However, it is typically easier to select the **Run/Continue Icon** from the main screen or to type the **F5** key.

Running and Stepping through Code

You can control the execution by choosing different modes.

"Run/Continue": Executes until (program ends / error occurs / user pause/ user stop)



"Single Step": Executes one assembly instruction

FAQ: I ran the code and encountered this "**Exception occurred at PC = 0x00000000**" error, what should I do?

Answer: Your QtSpim is not using address 0x00400000 as the default start address of your code. Click on "Simulator → Run Parameters" and change the value under "Address or label to start running program" from 0x00000000 to 0x00400000.

Setting Value into a

Setting Value into a Register

You may right click on a register in the register frame and select "Change Register Contents" to assign value directly into that register. Try to change the value of **R16** (which is **$16** or **$s0**) to decimal value **100** (or hexadecimal **64**).

```
R4  [a0] = 2
R5  [a1] = 7ffff54c
R6  [a2] = 7ffff558
R7  [a3] = 0
R8  [t0] = 0
R9  [t1] = 0
R10 [t2] = 0
R11 [t3] = 0
R12 [t4] = 0
R13 [t5] = 0
R14 [t6] = 0
R15 [t7] = 0
R16 [s0] = 0
R17 [s1] = 0
R18 [s2] = 0
R19 [s3] = 0
R20 [s4] = 0
```

Final Code:

- Can we write a simple C code and convert the same to MIPS??
- Yes. It is the next step to be done.

```
# include <stdio.h>
int main ()
{
    int a, b, c;
    c = a + b;
    printf ("\n Here you go", c);
    return 0;
}
```

```
.text    # the text section
.globl main # Can be called from anywhere.
main:
    ori $t0,$0,0x03
    ori $t1,$0,0x02
    add $t2,$t0,$t1
```